

Container Workload Characterization Through Host System Tracing

Madeline Janecek

Department of Computer Science
Brock University
St. Catharines, Canada
mj17th@brocku.ca

Naser Ezzati-Jivan

Department of Computer Science
Brock University
St. Catharines, Canada
nezzati@brocku.ca

Seyed Vahid Azhari

Department of Comp. and Soft. Engineering
Polytechnique Montreal
Montreal, Canada
seyed-vahid.azhari@polymtl.ca

Abstract—The use of containers within cloud environments has become increasingly popular due to their lightweight nature, scalability, and efficiency. However, as containers share their host's resources, advanced resource management techniques are essential to avoid performance impacting resource contention. Coarse measures such as CPU, disk, and network usage collected from internal agents are often considered, yet these methods may be improved upon to garner a more precise view of container workloads.

In this paper, we present a container workload characterization method using host system tracing. Features derived from thread execution states are taken from tracing data to reveal container runtime behaviour. A PageRank-based algorithm is then used to identify the most significant threads for further analysis. Once this data is collected and vectorized, a two stage K-Means clustering technique is used to generate groups of containers with similar workloads. This eliminates the need for manual analysis of individual containers, and instead allows administrators to view and address container behaviours collectively. Experimental results show that our methodology can identify a variety of execution behaviours. Administrators may use these results to remove idle containers to free up system resources. Moreover, they may identify clusters of containers that are at risk of resource contention, allowing for more effective resource assignment.

Index Terms—workload characterization, virtualization, performance analysis, containers, machine learning, K-Means

I. INTRODUCTION

Cloud computing is an emerging technology that replaces the need for active user management with flexible, on-demand access to shared resources. The use of containers to deploy applications within cloud environments, commonly referred to as Container-as-a-Service (CaaS), has grown increasingly popular over the last few years [1, 2]. In the Cloud Native Computing Foundation (CNCF)'s 2020 survey 92% of respondents reported they are currently using containers within their production environments, which is a 300% increase from just 2016 [3]. Containers provide several advantages, including high efficiency, scalability, and portability. However, despite all of their benefits containers are not without their own drawbacks. With container-based virtualization, containers share host resources like CPU, disk, and memory. As the number of concurrently running containers rises, so does the risk of resource contention, latencies and performance problems.

Consequently, advanced resource management solutions are essential to maintain high overall performance.

Many existing methods attempt to characterize container resource usage through coarse-grained metrics, such as CPU and memory usage, collected using internal tracking agents [4, 5]. However, internal access to containers within commercial cloud environments is often unavailable due to security and privacy policies. In the cases where it is permitted, the organization and maintenance of several internal agents can be challenging at a cloud-scale. These considerations motivated us to explore container workload characterization without the use of such internal agents. Instead, we turn to kernel execution tracing to obtain a more precise and detailed look at container execution behaviour. This method has the added benefit of being independent of the container's host. As such, it may be applied to containers running within a single host, a group of distributed nodes, within virtual machines in cloud environment, and so forth.

We also work to utilize techniques that efficiently characterize a large number of containers, as thousands of containers may be deployed on a single server [6]. We argue that employing clustering algorithms to group containers based on their runtime behaviour will reduce the complexity of large scale monitoring. By presenting problematic containers collectively rather than individually, administrators will be able to address issues on a larger scale. For instance, should one of the resulting clusters consist of idle containers, an administrator could consider stopping them to free up system resources. Furthermore, a cluster may group containerized applications that are suffering from resource contention. The administrator could respond by granting these applications access to more of the appropriate resource, or move some of the containers to another host.

In this paper, we propose a container workload characterization framework that utilizes unsupervised clustering techniques upon system level trace data. Firstly, we utilize a unified system tracing method to collect detailed execution information from the running containers. We then adapt the PageRank algorithm [7] to select each container's most relevant threads for further analysis. This ensures that inactive or inconsequential threads are not considered, resulting in a more consistent comparison between containers. Details regarding the execution

states of these threads are extracted, aggregated, and vectorized to create a single representation for each container. This data is then used to cluster the containers into distinct groups with a two stage clustering scheme using the popular K-Means algorithm. The silhouette score is employed to automatically determine the appropriate number of clusters. Experimental results show that this methodology will effectively identify both idle and intensive containerized processes.

The main contributions of this work are as follows: **First**, we present our host based feature extraction method using system tracing. **Second**, we explore a two stage clustering technique to characterize container workloads. **Third**, we evaluate the proposed approach on real data collected from containers running with a variety of workloads.

The remaining parts of this paper are as follows: Firstly, Section II discusses prior work related to the topic of cloud container performance analysis and execution tracing. Then Section III goes on to describe each step of our proposed methodology. Section IV discusses our experimental set up along with an analysis of its results. Section V then goes on to discuss the computational overhead of this approach. Finally, Section VI concludes this work.

II. RELATED WORK

One popular method within performance analysis is to collect metrics such as CPU, memory, disk, and file usages. In [4], L. Cai *et al.* collect similar metrics from running containers using open-source tools like cAdvisor and influxDB. They utilize this data along with clustering techniques to characterize a container's resource usage. The results are used to automatically configure parameters provided by commercial container management tools to limit resource usage. Their experimental results either meet or improve upon the container performance seen using the default configurations.

In another work, Zhang *et al.* [5] propose using density-based clustering along with performance related metrics to detect abnormal behaviour within large scale cloud platforms. They collect their performance metrics using BOSH agents. Despite their good results, both the previously mentioned works utilize internal resource monitoring agents to collect their data. Execution tracing is a non-intrusive method that may provide a more precise view of execution with minimal performance impact, making it better suited for complex and distributed systems [8].

Execution tracing is a well established tool for a variety of performance related analyses, including performance anomaly detection [9], root cause analysis [10], and program execution comprehension [11]. Tracing tools such as LTTng [12] record instrumentation points, commonly referred to as events, and store them in a trace file. These events may be taken from several different levels, such as the user-space, kernel, hardware, and network levels. Tracers are capable of collecting hundreds of thousands of events per second, resulting in a detailed record of overall execution while imposing very little overhead [13]. In [8], Giraleau *et al.* go on to describe how meaningful resource utilization metrics can be extracted from

these kernel traces. As a container's execution is handled by its host's operating system, we can use these tracing methods to extract fine-grained container resource usage metrics.

Other works have shown tracing's effectiveness for execution analysis within cloud environments. For example, Dean *et al.* [14] use kernel level tracing for anomaly detection in Infrastructure-as-a-Service (IaaS) clouds. Their statistical approach is capable of differentiating between external and internal anomalies. Additionally, their method identifies the top most affected system calls to provide fault localization assistance. Spillner [15] creates trace profiles of containers within cloud environments to measure their memory usage. This work is meant to help developers avoid financial penalties due to inaccurate memory allocation in cloud computing environments. In [16], Zhang *et al.* collect kernel event traces from cloud infrastructures. They then employ both data mining and statistical analysis tools to perform performance issue analysis. Considering the success seen in these works, tracing methods are shown to be a viable alternative to the aforementioned internal monitoring agents.

As the number of containers within a cloud environment increases, manual analysis becomes tedious and prone to errors. Consequently, several different works have explored the use of machine learning algorithms to automate container behaviour analysis. A survey of different works that explore the use of machine learning for resource management within cloud computing is presented by Goodarzy *et al.* in [17]. In [18] Ye *et al.* compare the effectiveness of Linear Regression (LR), Support Vector Machine (SVM) and Artificial Neural Network (ANN) as tools to model the execution behaviour of containerized applications. In [19], Ye *et al.* also explore support vector regression (SVR) as a performance prediction model of Spark within Docker containers. These techniques can achieve low prediction error, however they are not ideal for complex environments made up of a variety of applications.

Machine learning clustering techniques are an effective alternative for working within varied cloud environments. Several works have explored the use of these clustering techniques, including DBSCAN [5], Canopy clustering [4], k-nearest neighbours (KNN) [20], and k-medoids [4]. Nemati *et al.* [21] propose a VM workload characterization using a two stage K-Means clustering scheme to perform both course and fine grained workload characterization. They utilize host system tracing to collect detailed information on the underlying behaviour of running VMs. Considering the similarities between virtual machines and containers, we attempt to apply K-Means clustering within our own context.

To the best of our knowledge, there are no pre-existing works that conduct container workload characterization through the analysis of execution states taken from host system trace data. Through the use of tracing techniques, our framework is capable of extracting detailed information regarding container workloads with minimal overhead and simple deployment.

III. METHODOLOGY

In this section, we explain the methods we use to collect and interpret the system execution traces. A visual overview of the proposed pipeline is shown in Figure 1. Firstly, detailed information regarding each container's execution is collected using an open-source tracing tool. Then a ranking algorithm is utilized to select the most relevant threads of each container. Data regarding these threads' execution states is extracted and vectorized to represent their respective container. Finally, a two-stage clustering is used to compute coarse and fine-grained characterizations of the container workloads.

A. Background on Containers

Containers, similarly to virtual machines (VM), are a type of virtualization technology. Virtualization as a whole is a collection of software technologies that enables execution on virtual hardware or operating systems. The main difference between VMs and containers is each VM requires the installation of a guest operating system, as shown in Figure 2. Containers run on the host's operating system, and thus share its resources. They package all the necessary executables, libraries, and configuration details into a single unit for lightweight, efficient deployment of applications. For this reason, many developers have turned to containers to replace traditional VM virtualization in cloud environments [22].

Container engines are a type of software that assist users with container creation and management. There are several different commercial container engines, such as Docker Engine [23], RKT [24], CRI-O [25], and LXD [26]. A standard container engine accepts user input to create container instances from a file known as a container image. An engine is then used to call container runtimes. From a user perspective, a container engine may appear to handle container execution. However, from a technical standpoint, it is left to the host to handle the containerized application. Thus, it is not necessary to trace the container engine for our analysis.

B. Container Tracing

Linux Trace Toolkit next generation (LTTng) [12] is an open-source tracing tool used to collect events from multiple layers. Based on our observations, from a kernel perspective a process running within a container is no different than a process running on the host. As this is the case, we can use the raw kernel events collected by LTTng within our container analysis.

This unified tracing of containers is not reliant on internal access, and accordingly it is not affected by the privacy restrictions that are often present within cloud environments. It is also a more manageable alternative to utilizing resource monitoring agents with a large number of containers. However, this methodology does present the challenge of determining which threads are associated with a specific container. One solution is to specify certain metadata fields, commonly referred to as contexts, to be recorded at each trace event. A visualization of an event and its contexts is shown in Figure 3. The first context required for container aware tracing is

its namespace ID (`pid_ns`). A container has its own unique namespace. Therefore this context allows us to group threads into their respective container or even filter tracing by a specified container. Two other useful contexts are `tid` and `vtid`. The thread ID (`tid`) is a unique ID on the host, whereas the virtual thread ID (`vtid`) is an ID specific to the container's namespace. An example of setting up a tracing session which includes these contexts and only records a single container's events is shown in Figure 4.

C. Execution States

Using the kernel level trace events, we can extract detailed information on program execution. We accomplish this using the open-source visualization tool Eclipse Trace Compass [27]. Trace Compass provides several different builtin views to extract useful information from trace data. An example of one of these views is shown in Figure 5. The Trace Compass Scripting plugin also adds scripting capabilities for flexible data analysis. We utilize JavaScript within this scripting environment to perform the analyses described in the following sections.

Based on observations taken from our kernel trace analysis, we saw that a thread can run in several different states throughout its execution. A visualization of these kernel execution states, along with what prompts a state switch, is shown in Figure 6. Two of these states, which we refer to as "user mode" and "system call mode", indicate that the thread is executing code. Accordingly, we refer to these two states as "running" states. They differ in the level of privileges granted to the thread. User mode is associated with user application code, and thus does not have high level privileges. Threads executing in system call mode have higher privileges, and thus are capable of executing kernel functions. We also consider four "blocking" states, which are distinguishable by the cause of their blocked status. These execution states include "preempted", "interrupted", "blocked", and "waiting".

A thread is said to be in a preempted state when its ready for execution, but is forced to wait because of a lack of assignable CPU time slices. This situation could arise when a higher priority thread is running, or if the current thread has simply run the duration of its time slice. A thread would enter an interrupted state when an interrupt request (IRQ) occurs. Such a thread would remain in this interrupted state while the interrupt handling is performed. A thread is put into a blocked state when it must wait for some condition to be met before it may continue its execution. An example of this is when a thread must wait for the allocation of additional resources, or when another thread must complete its tasks. Finally, there are times when a thread is not active. For instance, this would be the case if a thread has been created but has not started execution. In this case, the thread is said to be in a waiting state.

Certain kernel events can be used to identify context or changes in thread execution state [28]. For instance, when an `irq_softirq_entry` or `irq_handler_entry` event occurs, this indicates that the thread has entered an inter-

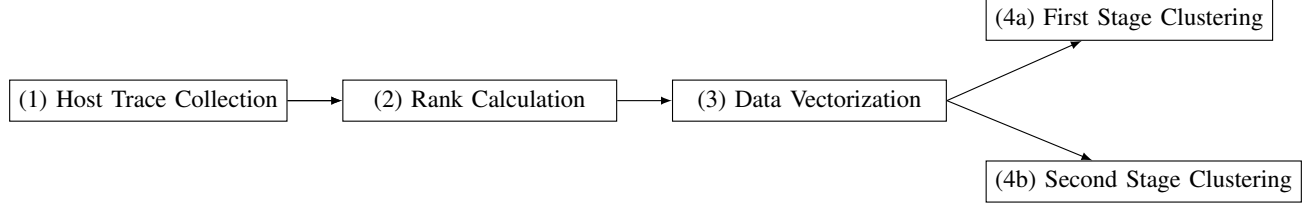


Fig. 1. The proposed pipeline for workload characterization.

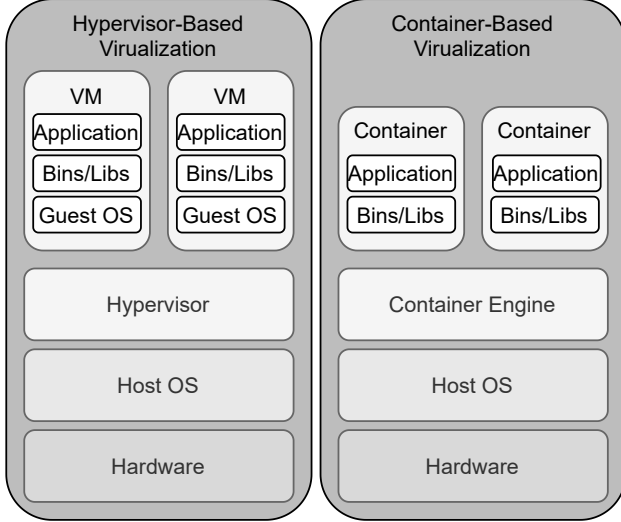


Fig. 2. Comparison between virtual machines and containers.

```

[11:46:35.255401558] (+0.000011922) context_tracing
syscall_entry_futex: { cpu_id = 0 }, { pid = 2313, vpid =
1, tid = 2379, vtid = 38, pid_ns = 4026532249 }, {}
  
```

Fig. 3. A tracing event that includes the pid_ns context.

rupted state. It will leave this interrupted state when an `irq_softirq_exit` or `irq_handler_exit` event takes place. The `sched_waking` event shows that a thread is ready for execution should a time slice become available, and as such this event indicates that the thread is in a preempted state. The `sched_switch` event can be indicative of different state switches depending on the thread's state prior to the switch. For example, should some the thread j be in a running state, then the `sched_switch` event indicates that j was replaced by some other thread k . Therefore thread j goes into a preempted state and thread k enters a running state. The `sched_switch` event can similarly identify blocked and waiting state switches. The pseudocode for using these events to determine a thread's execution state is shown in Algorithm 1.

By extracting details pertaining to each thread's state sequence, we can characterize the type of application workload

```

# Get process id of the container's init
process
> pid=$(docker inspect --format '{{.State.
Pid}}' container_name)

# Get the container's namespace
> namespace=$(lsns -n -t pid -o NS -p ${
pid})

# Create tracing session
> lttng create session_name

# Add necessary contexts to tracing
session
> lttng add-context -k -t pid_ns -t tid -t
vtid -t procname -t pid -t vpid

# Enable all syscall events and filter by
container's namespace
> lttng enable-event -k --syscall -a --
filter='\'$ctx.pid_ns == ${namespace
}\''
  
```

Fig. 4. Arguments to set up a LTTng tracing session set to only collect the events of a single Docker container.

within a container. For example, if a container's threads are frequently entering the blocked state, this is indicative of I/O intensive operations. Furthermore, if the threads are rapidly switching into preempted, user mode, and system call mode states, this is indicative of CPU intensive operations. A complete list of the metrics we extract from the thread state sequences is shown in Table I.

D. Thread Ranking Algorithm

A single running container is often made up of several active threads. The total number of threads varies from case to case depending on the type of containerized application and its workload. Based on our observations, even idle containers may contain multiple threads. For instance, one idle container containing Apache ZooKeeper still contained thirty threads. Of these thirty threads, only three exhibited behaviour worth analyzing. The remaining twenty seven threads remained in the waiting state throughout the entire tracing session, and thus were not worth any further consideration. To make meaningful comparison between containers, we must generate a single standardized representation. We propose that first identifying

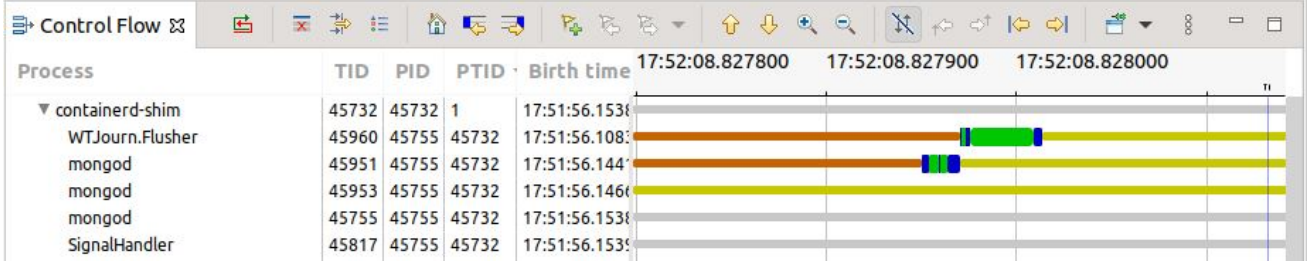


Fig. 5. Trace Compass's Control Flow view.

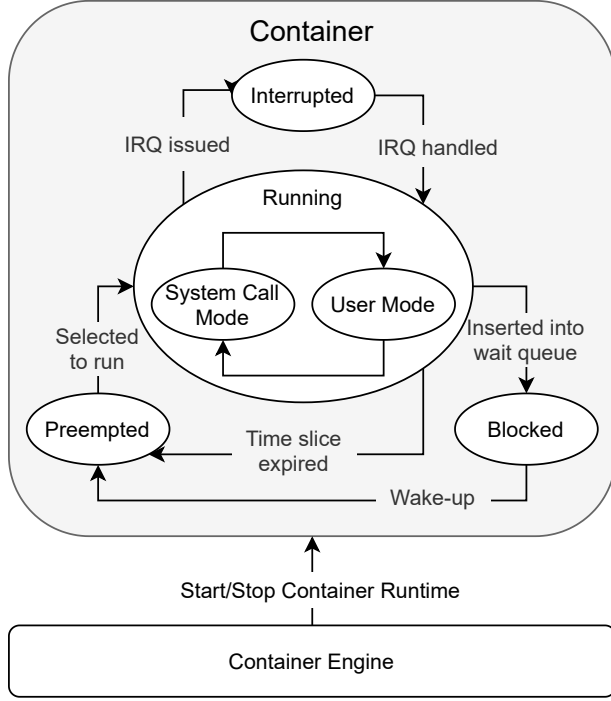


Fig. 6. Execution states of active threads.

a container's top most significant threads and then combining their metrics is an effective solution.

To identify a container's most significant threads, we adapt the PageRank (PR) algorithm [7]. This algorithm is most commonly used in web structure mining to find pertinent web pages to a given query. It works by counting both the quantity and quality of links pointing to each web page. A page which has many links from highly ranking pages will be given a high rank. For our purposes, the modified ranking algorithm utilizes the inter-thread interactions to compute a thread relevancy score. The pseudocode for this ranking algorithm is shown in Algorithm 2.

Inspired by methods presented in [29], we start by creating a directed graph where each node represents an individual thread. Directed edges between nodes are then generated to denote the interactions between the corresponding threads. Any

Algorithm 1: Blocked Reason Analysis Algorithm

```

if  $event == sched\_waking$  then
    |  $thread_j^{state} = preempted;$ 
end
else if  $event == irq\_softirq\_entry$  or  $event ==$ 
     $irq\_handler\_entry$  then
    |  $thread_j^{state} = interrupted;$ 
end
else if  $event == sched\_switch$  then
     $prev\_state = getEventField(event, "prev\_state");$ 
    if  $prev\_state == TASK\_DEAD$  or  $prev\_state ==$ 
         $WAKEKILL$  then
        |  $thread_j^{state} = waiting;$ 
        |  $thread_k^{state} = thread_k^{last\_running\_state};$ 
    end
    else if  $prev\_state ==$ 
         $TASK\_INTERRUPTIBLE$  then
        |  $thread_j^{state} = blocked;$ 
        |  $thread_k^{state} = thread_k^{last\_running\_state};$ 
    end
    else if  $prev\_state == TASK\_RUNNING$  then
        |  $thread_j^{state} = preempted;$ 
        |  $thread_k^{state} = thread_k^{last\_running\_state};$ 
    end
end
else
    |  $thread_j^{state} = waiting;$ 
end

```

Algorithm 2: Thread Ranking Algorithm

```

for  $event \in trace$  do
    if  $event == sched\_wakeup$  then
        |  $wakerTID = getWakerTID();$ 
        |  $wakeeTID = getWakeeTID();$ 
        |  $incrementLinkCount(wakerTID);$ 
        |  $link(wakeeTID, wakerTID);$ 
    end
end
for  $node \in graph$  do
    |  $updateRank(node);$ 
end

```

TABLE I
METRICS EXTRACTED FROM TRACE DATA

Metric	Description
D_{priv}	Percentage of time performing privileged execution
D_{non}	Percentage of time performing non-privileged execution
$W_{unknown}$	Percentage of time where wait reason is unknown
W_{irq}	Percentage of time waiting for interrupt handling
W_{res}	Percentage of time waiting for resources
W_{cpu}	Percentage of time waiting for CPU
F_{priv}	Frequency of privileged execution
F_{non}	Frequency of non-privileged execution
F_{disk}	Frequency of wait for resources
F_{irq}	Frequency of interrupt request injection
$F_{preempt}$	Frequency of preemption
$F_{inactive}$	Frequency of inactive status

inter-thread dependencies may be used to identify these linkages, but in this work we primarily use the `sched_wakeup` event. Upon completing this graph, we move on to compute the rank of each thread. We let the set s consist of all threads that point to a thread p . Then we let q be a thread within s , and n be the number of links from q . The rank (R) of p is computed as:

$$R(p) = \sum_{q \in s} \frac{R(q)}{n} \quad (1)$$

In this work, the rank of each node is initially set to $\frac{1}{t}$, where t is the total number of threads recorded in the trace data. A simplified visualization of an execution graph where the initial ranks are set to 1 is shown in Figure 7. In this example, Thread A is said to be a very significant thread. It has the greatest number of edges pointing to it, which results in a higher rank. Thread C and Thread E have the same number of links pointing to them, however they have different ranks. This is because a link pointing from Thread F is considered to be more significant as it interacts with fewer threads than Thread D. As such, Thread E has a higher rank than Thread C.

E. Vectorization

Most machine learning algorithms expect fixed sized vectors as input. Consequently, the collected execution trace data's raw, semi-structured format is not suitable for this kind of automated analysis. Thus, the collected performance metrics we extract must be vectorized in a data preprocessing phase. In this work, we propose generating a *state duration vector* to represent each container. With these duration vectors, there is an entry for each of the six possible execution states. The value of each entry represents the total amount of time the container's top most relevant threads entered the corresponding state, i.e.:

$$C_i = (s_{i1}, s_{i2}, s_{i3}, s_{i4}, s_{i5}, s_{i6}) \quad (2)$$

We also normalize the input vectors to improve upon the quality and efficiency of comparison. To do this, we utilize

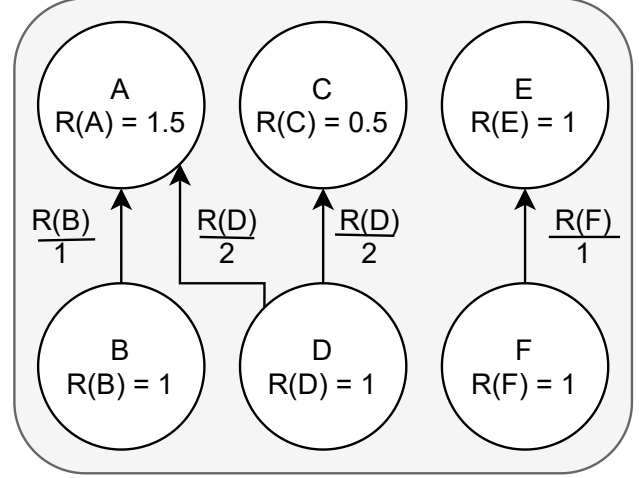


Fig. 7. A visualization of ranking threads when the initial ranks were set to 1.

the $L2$ norm [30]. This results in a set of input vectors with the same magnitude, i.e.:

$$\sum_{m=1}^M f_{im}^2 = 1, \forall i = 1..N \quad (3)$$

F. Container Workload Clustering

When selecting the method to group containers based on their their execution behaviour, two main factors influenced our decision. Firstly, the algorithm could not be reliant on labeled training data. This information may not be readily available, and it would constrict our approach's ability to discover new execution behaviours. As such, supervised machine learning classifiers were not viable options. Secondly, the approach had to characterize all entries. This meant that popular clustering algorithms that automatically identify outliers, such as DBSCAN [31] or OPTICS [32], were not suitable for this work.

With this in mind, we chose to group the state count vectors using the popular K-Means clustering algorithm [33]. As an unsupervised clustering algorithm, this method has the added advantage of not requiring labeled training data. As such, the proposed methodology is not confined to user supplied definitions. The K-Means algorithm works by identifying k centroids, which serve as the cluster centers. In our analysis, a centroid is used as a single representation of the various workloads found within the cluster. A distance function is then used to assign each of the entries to their spatially closest cluster. This is done with the overarching goal of maximizing the intra-cluster similarity as well as the dissimilarity between clusters, i.e.:

$$\min \sum_{i=1}^N \sum_{k=1}^M (f_{ik} - c_{jk})^2 \quad (4)$$

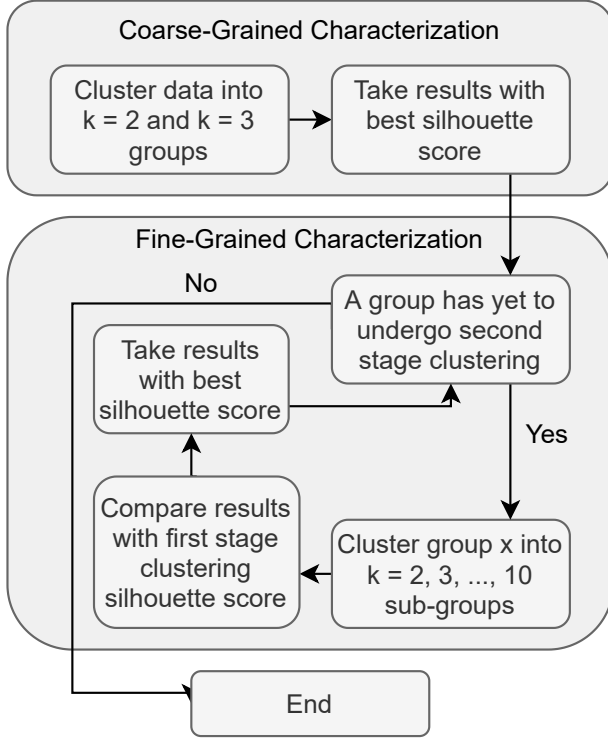


Fig. 8. Two stage K-Means clustering scheme.

One of the greatest disadvantages of K-Means clustering is the number of clusters k must be declared beforehand. The optimal value of clusters may vary widely from case to case, and as such k should not be set to a static number. It is also not a value that can be easily determined by the user. To maintain the automatic nature of our approach, we propose using the silhouette coefficient (5) to determine the optimal value of k . This metric measures the similarity of entries within their own clusters compared to that of other clusters. This results in a value ranging from -1 to +1, where a high positive value indicates highly separated clustering. We suggest clustering the data multiple times with a range of k values and selecting the results with the highest silhouette score. K-means clustering is highly efficient, even with a large dataset, and as such repeated clustering does not significantly impact the effectiveness of the proposed approach.

$$s_i = \frac{out_i - in_i}{\max(in_i, out_i)} \quad (5)$$

We employ a two stage K-Means clustering approach to provide a more detailed container workload characterization. This provides coarse-grained characterization at the first stage of clustering, as well as a finer-grained characterization of the various containers. At the first stage of clustering, we apply K-Means clustering where k equals 2 and 3. The clustering results with the highest silhouette score are used to provide a coarse-grained characterization of the container workloads. While an optimal silhouette score may be found using higher k

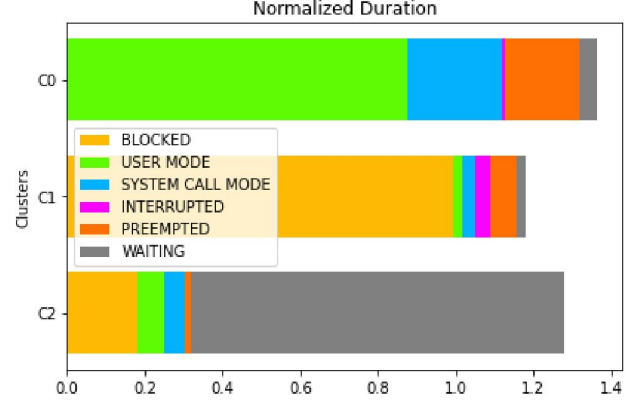


Fig. 9. Workload characteristics taken from first stage clustering centroids.

values, we limit the number of clusters at this step to maintain the manageability of our approach. We then cluster each of the resulting clusters to get the finer-grained characterization of the container workloads. To do this, we repeatedly apply K-Means clustering on each of the clusters using k values ranging from 2 to 10 (inclusive), and take the result with the highest silhouette score. A visual overview of this process is shown in Figure 8.

IV. EVALUATION

In this section, we present the experimental results obtained with the proposed approach. At each step, we describe the attributes of the resulting clusters, as well as what an administrator would be able to gain from the results in a real life scenario.

A. Experimental Setup

These following experiments were conducted using Ubuntu 20.04.1 with Linux kernel 5.8.0. Each of the containers were created using Docker 20.10.4. We utilized different benchmarking tools, such as ApacheBench and Sysbench, to ensure that a variety of workloads, including network intensive, CPU intensive, disk I/O intensive, and idle workloads were present in the test data.

B. First Stage Clustering

The first stage of clustering split the containers into three groups with an overall silhouette score of 0.7945. These clusters will be referred to as C0, C1, and C2. The normalized metrics taken from each of the cluster centroids is shown in Figure 9. When manually looking at the contents of first cluster (C0), we observed that it primarily consisted of containers with CPU intensive workloads. This is reflected in the performance metrics extracted using the proposed method, as on average items within this cluster spent 97.3% of their execution time within running or preempted states. When making a comparison between clusters, on average the items within C0 spent 881.9% more time in these states than those within C1, and 37947.1% more time than the items within C2.

Looking at the entries within the third (C2) cluster revealed that they mostly consisted of containers that were running idle. These containers spent significantly less time within active states, and instead spent 73.4% of their execution time within the waiting state. As the number of containers rise, it can become increasingly difficult to determine which are unnecessarily using up resources. In a real scenario, an administrator may consider stopping the containers within cluster C2 to release system resources.

The remaining items made up the second (C1) cluster, and the majority of these containers had network or I/O intensive workloads. This reflected in the extracted metrics as 85.2% of their execution duration spent within the blocked state. A breakdown of each cluster's average execution time in each state is shown in Table II.

TABLE II
AVERAGE DURATION IN EACH STATE (MS)

State	C0	C1	C2
User Mode	668.72	18.17	1.15
System Call Mode	150.83	29.15	1.06
Blocked	0.00	849.27	6.26
Interrupted	5.55	30.03	0.04
Preempted	140.48	50.45	0.31
Waiting	26.42	19.52	21.47

C. Second Stage Clustering

After observing these coarse-grained results, we applied K-Means clustering to the three groupings. With this experimental set up, the items in cluster C2 were too similar, and the overall silhouette score would not improve with further clustering. This being the case, it remained as one cluster. Both C0 and C1 were divided into two subgroups, which will be referred to as clusters C0-0, C0-1, C1-0 and C1-1. This clustering resulted in an overall silhouette score of 0.6527. Each of these clusters' average execution time within the execution states is shown in Table III.

TABLE III
AVERAGE DURATION IN EACH STATE (MS)

State	C0-0	C0-1	C1-0	C1-1
User Mode	800.78	448.62	50.60	8.44
System Call Mode	6.32	391.68	39.48	26.06
Blocked	0.00	0.00	706.06	892.23
Interrupted	5.41	5.79	63.52	19.98
Preempted	174.70	83.46	127.02	27.48
Waiting	0.03	70.41	9.49	22.52

The workload characteristics taken from the centroids of cluster C0's subgroups is shown in Figure 10. Entries within C0 are all containers with CPU intensive workloads that spent the majority of their execution time within running states. However, members of the two groups differ in the level of privileges necessary to perform their assigned operations. Items within C0-0 spent most of their time executing user application code, whereas items within C0-1 were performing

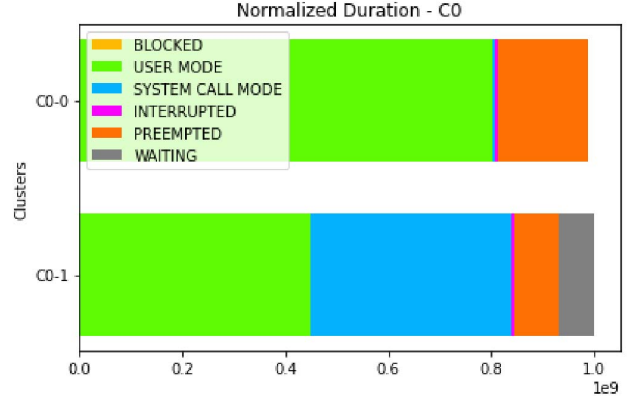


Fig. 10. Workload characteristics discovered from second stage clustering centroids.

higher level operation. Consequently, on average 46.6% of the time spent in running states is within system call mode. This is significantly higher than C0-0, which on average spent only 0.78% of its time in running states within system call mode. We observed that the second stage of clustering split cluster C1 into network intensive (C1-0) and file I/O intensive (C1-1) subgroups. The workload characteristics taken from these two clusters are shown in Figure 11.

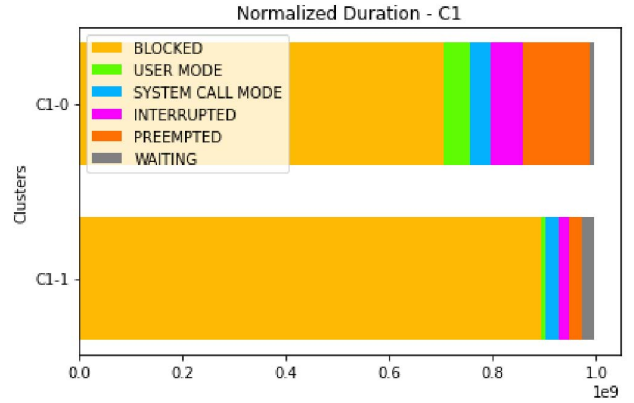


Fig. 11. Workload characteristics discovered from second stage clustering centroids.

In summary, the proposed method was able to cluster the container workloads to generate five distinct groups based on their execution behaviour. These clusters consisted of idle containers, disk I/O intensive workloads, network intensive workloads, CPU intensive workloads, and privileged CPU intensive workloads.

Commercial containerization products often accept parameters to limit the application's resource usage. This is to mitigate the high risk of resource contention. However, it can be challenging to determine the optimal values for these parameters, especially with a large number of containers. This fine-grained clustering stage may be used to characterize the

container workloads, giving administrators a better sense as to the optimal parameter configurations for a given cluster. Administrators might also use these results to identify which containers are currently suffering from resource contention, and respond by allocating more of the specific resources to the impacted containers.

V. OVERHEAD ANALYSIS

In this section, we discuss the computational overhead involved with the proposed approach. Specifically, we examine the cost of system-level tracing along with data clustering.

A. Tracing

To measure the tracing overhead we considered three different tracing configurations, including:

- No Tracing: we record the application's performance without using system tracing.
- Full Tracing: we perform system tracing with all kernel tracepoints enabled. This configuration represents the worst case scenario, and is not necessary for the proposed method. Instead, it is included for the purpose of comparison.
- Minimal Tracing: we only enable the kernel tracepoints that are necessary for our proposed method. The required tracepoints for this configuration are listed in Table IV.

TABLE IV
TRACEPOINTS REQUIRED FOR FEATURE EXTRACTION

Tracepoint	Description
sched_waking	Thread wakes up and resumes a task
sched_wakeup	Event we use to indicate inter-process signaling for thread ranking
irq_softirq_entry	Execution of software interrupt handler begins
irq_softirq_exit	Execution of software interrupt handler begins
irq_irqhandler_entry	Execution of hardware interrupt handler begins
irq_irqhandler_exit	Execution of hardware interrupt handler ends
sched_switch	Thread is scheduled in or out

Table V shows the execution duration of each configuration. The results were taken from the average of 20 executions of common command line utilities, and will be used as an indication of computational overhead. The results show that tracing all kernel events imposed a 21.6% overhead. This can be improved by limiting tracing to the events related to our analysis. With this configuration, tracing only imposes a 3.6% overhead.

B. Clustering

To test the efficiency and effectiveness of the two stage K-Means clustering approach, we compare its results to that of two other commonly used clustering methods. Specifically, we applied spectral clustering as well as mean shift clustering

TABLE V
DURATION INDICATING TRACING OVERHEAD

Benchmark	Duration (ms)	Overhead
No Tracing	214.85	0.00%
Full Kernel Tracing	261.30	21.62%
Minimal Tracing	222.60	3.61%

to the same data. Spectral clustering, similarly to K-Means clustering, requires that the number of clusters is declared prior to execution. Thus, for K-Means and spectral clustering, we say that one execution entails clustering the data multiple times to discover the optimal number of clusters. Mean shift clustering does not require this parameter, so the results of one execution are taken from clustering the data only once. The results shown in Table VI are from the average of 10 executions.

TABLE VI
AVERAGE DURATION AND QUALITY OF CLUSTERING ALGORITHMS

Algorithm	Duration (seconds)	Silhouette Score (%)
K-Means Clustering	0.67	95.47
Spectral Clustering	14.68	95.04
Mean Shift Clustering	0.85	95.02

These results indicate that out of the three clustering algorithms, K-Means is the most efficient and effective for our approach. With a silhouette score of 95.47%, it is an improvement on score seen with both spectral clustering and mean shift clustering. On average, it is significantly faster runtime than spectral clustering. Additionally, despite having to cluster the data multiple times, the K-Means clustering's average runtime is 78.34% the average runtime of mean shift clustering.

VI. CONCLUSIONS AND FUTURE WORK

The use of containers is becoming increasingly popular within cloud environments, despite their complex troubleshooting and debugging. The process of monitoring thousands of concurrently running containers could be significantly simplified by automatically grouping containers based on workload characterizations. Administrators could use such methods to deal with issues collectively rather than dealing with them on an individual basis.

In this paper, we presented a host based workload characterization framework utilizing unified tracing of containers. State information from each container's most relevant threads is extracted and converted into a single vector representations. The containers are then clustered using a two stage K-Means clustering scheme with the silhouette coefficient to determine the optimal number of clusters. Using experimental results, we show how this method may be used to identify both idle and intensive containers on a cloud-scale while imposing only minimal overhead.

The results of the proposed method may be used to model resource allocations as well as make optimal configuration

suggestions to administrators, which will be considered in future work. Future work will also include examining how different input and tasks may affect clustering. Moreover, we can improve upon our current technique by extracting other features from the system trace data. For example, differentiating between different types of interrupts instead of representing them collectively may give a more in depth picture of execution. Additionally, future work may consider more advanced deep learning techniques to characterize container workloads.

REFERENCES

- [1] A. Alimudin and Y. Ishida, "Service-based container deployment on Kubernetes using stable marriage problem," in *Proceedings of the 2020 The 6th International Conference on Frontiers of Educational Technologies*, ser. ICFET 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 164–167. [Online]. Available: <https://doi.org/10.1145/3404709.3409389>
- [2] B. Ruan, H. Huang, S. Wu, and H. Jin, "A performance study of containers in cloud environment," vol. 10065, 11 2016, pp. 343–356.
- [3] "Cloud native survey," 2020. [Online]. Available: https://www.cncf.io/wp-content/uploads/2020/11/CNCF_Survey_Report_2020.pdf
- [4] L. Cai, Y. Qi, W. Wei, and J. Li, "Improving resource usages of containers through auto-tuning container resource parameters," *IEEE Access*, vol. 7, pp. 108 530–108 541, 2019.
- [5] X. Zhang, F. Meng, and J. Xu, "PerfInsight: A robust clustering-based abnormal behavior detection system for large-scale cloud," in *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*, 2018, pp. 896–899.
- [6] P. Ghosh, Q. Nguyen, and B. Krishnamachari, "Container orchestration for dispersed computing," in *Proceedings of the 5th International Workshop on Container Technologies and Container Clouds*, ser. WOC '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 19–24. [Online]. Available: <https://doi.org/10.1145/3366615.3368354>
- [7] W. Xing and A. Ghorbani, "Weighted PageRank algorithm," 01 2004, pp. 305–314.
- [8] F. Giraldeau, J. Desfossez, D. Goulet, M. Desnoyers, and M. R. Dagenais, "Recovering system metrics from kernel trace," in *Linux Symposium 2011*, Jun. 2011.
- [9] I. Kohyanejadfard, M. Shakeri, and D. Aloise, "System performance anomaly detection using tracing data analysis," in *Proceedings of the 2019 5th International Conference on Computer and Technology Applications*, ser. ICCTA 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 169–173. [Online]. Available: <https://doi.org/10.1145/3323933.3324085>
- [10] C. Biancheri, N. E. Jivan, and M. R. Dagenais, "Multilayer virtualized systems analysis with kernel tracing," in *2016 IEEE 4th International Conference on Future Internet of Things and Cloud Workshops (FiCloudW)*, 2016, pp. 1–6.
- [11] B. Cornelissen, A. Zaidman, A. Deursen, L. Moonen, and R. Koschke, "A systematic survey of program comprehension through dynamic analysis," *Software Engineering, IEEE Transactions on*, vol. 35, pp. 684 – 702, 11 2009.
- [12] M. Desnoyers and M. Dagenais, "LTTng: Tracing across execution layers, from the hypervisor to user-space," in *Linux Symposium*, 2008, p. 101.
- [13] L. Prieur-Drevon, R. Beamonte, and M. Dagenais, "R-sht: a state history tree with r-tree properties for analysis and visualization of highly parallel system traces," *Journal of Systems and Software*, vol. 135, 09 2017.
- [14] D. J. Dean, H. Nguyen, P. Wang, and X. Gu, "PerfCompass: Toward runtime performance anomaly fault localization for infrastructure-as-a-service clouds," in *Proceedings of the 6th USENIX Conference on Hot Topics in Cloud Computing*, ser. HotCloud'14. USA: USENIX Association, 2014, p. 16.
- [15] J. Spillner, "Resource management for cloud functions with memory tracing, profiling and autotuning," in *Proceedings of the 2020 Sixth International Workshop on Serverless Computing*, ser. WoSC'20. New York, NY, USA: Association for Computing Machinery, 2020, p. 13–18. [Online]. Available: <https://doi.org/10.1145/3429880.3430094>
- [16] H. Zhang, J. Rhee, N. Arora, S. Gamage, G. Jiang, K. Yoshihira, and D. Xu, "CLUE: System trace analytics for cloud service performance diagnosis," in *2014 IEEE Network Operations and Management Symposium (NOMS)*, 2014, pp. 1–9.
- [17] S. Goodarzy, M. Nazari, R. Han, E. Keller, and E. Rozner, "Resource management in cloud computing using machine learning: A survey," in *2020 19th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2020, pp. 811–816.
- [18] K. Ye, Y. Kou, C. Lu, Y. Wang, and C.-Z. Xu, "Modeling application performance in Docker containers using machine learning techniques," 12 2018, pp. 1–6.
- [19] K. Ye and Y. Ji, "Performance tuning and modeling for big data applications in Docker containers," in *2017 International Conference on Networking, Architecture, and Storage (NAS)*, 2017, pp. 1–6.
- [20] Z. Li, Z. Zhang, X. Liu, and C. Zhu, "Anomaly detection for container cluster based on JointCloud platform," in *Proceedings of the 2019 3rd International Conference on Compute and Data Analysis*, ser. ICCDA 2019. New York, NY, USA: Association for Computing Machinery, 2019, p. 26–30. [Online]. Available: <https://doi.org/10.1145/3314545.3314567>
- [21] H. Nemati, S. V. Azhari, and M. R. Dagenais, "Host hypervisor trace mining for virtual machine workload characterization," in *2019 IEEE International Conference on Cloud Engineering (IC2E)*, 2019, pp. 102–112.

- [22] Q. Zhang, L. Liu, C. Pu, Q. Dou, L. Wu, and W. Zhou, "A comparative study of containers and virtual machines in big data environment," 07 2018, pp. 178–185.
- [23] "Docker Engine." [Online]. Available: docker.com
- [24] "RKT." [Online]. Available: <https://coreos.com/rkt/>
- [25] "CRI-O." [Online]. Available: <https://cri-o.io/>
- [26] "LXD." [Online]. Available: <https://linuxcontainers.org/lxd/>
- [27] "Eclipse Trace Compass," 2021. [Online]. Available: <https://projects.eclipse.org/projects/tools.tracecompass>
- [28] N. Ezzati-Jivan, Q. Fournier, M. Dagenais, and A. Hamou-Lhadj, "DepGraph: Localizing performance bottlenecks in multi-core applications using waiting dependency graphs and software tracing," 09 2020, pp. 149–159.
- [29] J. Wang and N. Ezzati-Jivan, "Enhanced execution trace abstraction approach using social network analysis methods," in *2020 11th Symposium on Software Performance*, Aug 2020.
- [30] I. S. Dhillon, Y. Guan, and B. Kulis, "Kernel K-Means: Spectral clustering and normalized cuts," in *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD '04. New York, NY, USA: Association for Computing Machinery, 2004, p. 551–556. [Online]. Available: <https://doi.org/10.1145/1014052.1014118>
- [31] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, ser. KDD'96. AAAI Press, 1996, p. 226–231.
- [32] M. Ankerst, M. M. Breunig, H.-P. Kriegel, and J. Sander, "Optics: Ordering points to identify the clustering structure," in *Proceedings of the 1999 ACM SIGMOD International Conference on Management of Data*, ser. SIGMOD '99. New York, NY, USA: Association for Computing Machinery, 1999, p. 49–60. [Online]. Available: <https://doi.org/10.1145/304182.304187>
- [33] J. McQueen, "Some methods for classification and analysis of multivariate observations," *Computer and Chemistry*, vol. 4, pp. 257–272, 01 1967.